



Docker — Day 1

Hands-on notes from my first real practice session — what I did, what broke, and why

[Concepts](#)[Setup](#)[Real commands](#)[Gotchas I hit](#)[Ports deep-dive](#)

How to read this: normal boxes explain what I did. **Purple "Bonus detail" boxes** are extra things I should know. **Red "Gotcha" boxes** are mistakes I actually made. **Green "Added" boxes** are topics from later in the session.

1 The Big Idea (recap)

Docker packages an app *together with everything it needs to run* into a box called a **container**, so it runs the same way on any machine. It solves the classic *"it works on my machine"* problem.

The three words everything comes back to

Word	What it is	Analogy
Dockerfile	The recipe — instructions to build the environment	A written recipe
Image	The packaged result of the recipe — a frozen snapshot	A sealed ready-meal
Container	A running instance of an image — the live app	The meal, heated & being eaten



The single most important distinction

Image = the recipe-result (the box on the shelf). Container = a running copy of it. One image can create many containers. Almost every confusion today traced back to mixing these two up.

Sandbox = each app's own sealed box

Each container is a **sandbox**: it has its own filesystem, libraries, network and memory. An app inside can't see or break anything outside it. So two apps needing different versions of the same tool run side

by side without conflict, and deleting a container leaves nothing behind on my real machine.

2 Setup — the Engine & the Daemon

My very first command failed:

```
docker run hello-world
failed to connect to the docker API at unix:///.../docker.sock ...
the daemon ... no such file or directory
```

Why it failed

The `docker` command is just a **messenger (the "client")**. It needs a running **engine (the "daemon")** to talk to. On a Mac that engine lives inside **Docker Desktop**, which has to be open. The error meant: "the engine isn't running."

The fix:

```
open -a Docker           # launch Docker Desktop (wait for the 🐳 in the menu bar to
settle)
docker info              # confirm the engine is up – prints a Server section when
ready
```

💡 Bonus detail — the client/server split

Docker has two halves: the **client** (the `docker` command I type) and the **daemon/server** (the background engine that actually builds images and runs containers). They talk over a "socket" (`docker.sock`). On Mac/Windows the daemon runs inside a tiny Linux VM that Docker Desktop manages — that's why `docker info` showed `OSType: linux` even though I'm on a Mac.

3 Docker Hub & Ready-Made Images

I ran a real web server (**nginx**) without installing or writing anything:

```
docker run -d -p 8080:80 --name roshanprac nginx
```

It worked because I didn't *make* nginx — I **downloaded a ready-made image** of it. **nginx** is a famous, widely-used web server; its team already built an image and published it to **Docker Hub** — a giant public library of ready-to-run images.



Docker Hub = a shelf of ready-meals

nginx is a meal someone already cooked and sealed. `docker run nginx` = grab it and heat it. I run a finished product without cooking anything. **Running ≠ building.**

How the run actually decides what to do:

```
docker run nginx
→ Do I have the nginx image locally?
  → No → pull it from Docker Hub, then run
  → Yes → just run it (no download)
```



Bonus detail — what gets downloaded

Docker Hub stores the **built image**, not the Dockerfile. The nginx team used a Dockerfile *once* to build the image; I download the finished box. Also — my own images build *on top of* ready-made ones: my Step-2 Dockerfile starts with `FROM node:20-alpine`, meaning "start from the ready-made Node image, then add my code." Ready-made and custom images aren't separate worlds.

4 Ports — `-p 8080:80`

A **port** is like an apartment number. The address (`localhost`) gets you to the building; the port says which program to knock on. One machine runs many programs, each on a different port.

Port 80 is the worldwide standard port for normal web traffic (HTTP). Browsers assume it automatically — that's why you never type `:80` on real websites. nginx listens on 80 because it's a web server.

```
-p 8080:80
```

```
├──┬── 80    = port INSIDE the container (where nginx listens – the web standard)
└──┴── 8080 = port on MY Mac that forwards into it
```

```
Browser → localhost:8080 → [wall] → container:80 → nginx answers
```

Read it as **host : container** — "pipe my Mac's 8080 into the container's 80." I can map any outside port I like; the inside stays 80:

```
docker run -d -p 9999:80 --name roshanprac nginx # now answers at localhost:9999
```

Port	Standard use
80	HTTP (normal web)
443	HTTPS (secure web)
6379	Redis
5432	PostgreSQL
3306	MySQL

💡 Bonus detail — why 8080 and not 80?

Ports below 1024 are "privileged" and can need special permission or clash with other services, so for local practice people use **8080**. With `-p 80:80` you could visit plain **localhost** with no port number.

What if I leave OUT the port (and the name)?

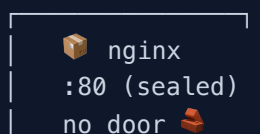
The name and the port behave **completely differently** when omitted — this surprised me:

✅ Added — a missing name auto-fills, a missing port does NOT

Skip `--name` → Docker invents a random name (e.g. *magical_franklin*); no problem. Skip `-p` → there is **no port mapping at all**. Docker does *not* pick one for me. nginx still starts and still listens on port 80 *inside* the container, but **no doorway is punched through to my Mac**, so **localhost:anything** → *Connection refused*.

WITHOUT `-p`:

Your Mac



WITH `-p 8080:80`:

Your Mac



localhost → refused

localhost:8080 → works

This is the **sandbox** idea again: a container's ports are private by default. `-p` is what **publishes** (exposes) a port — it punches the doorway. No `-p` = no doorway = unreachable from the browser.

The automatic option: capital `-P`

If I *do* want ports handled automatically, capital `-P` publishes all the image's declared ports, each to a **random** free port on my Mac:

```
docker run -d -P nginx
docker ps          # PORTS column shows e.g. 0.0.0.0:55001->80/tcp → visit
localhost:55001
```

Flag	Port behavior
(nothing)	No mapping — container is unreachable from my Mac
<code>-p 8080:80</code>	I choose: Mac's 8080 → container's 80
<code>-P</code> (capital)	Docker picks a random Mac port → container's declared ports

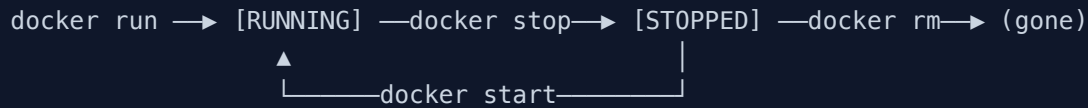


Proof to run

`docker run -d --name test1 nginx` then `curl localhost:80` → *refused* (the PORTS column is empty). Then `docker run -d -P --name test2 nginx` → PORTS shows a random mapping that works. **The PORTS column IS the doorway, made visible.**

5 The Container Lifecycle

A container moves through states. Key insight: **stopping is not deleting**.



Command	What it does
<code>docker run img</code>	Create + start a new container from an image
<code>docker ps</code>	List running containers only
<code>docker ps -a</code>	List all containers (incl. stopped) — <code>-a</code> = all
<code>docker logs name</code>	Show a container's output
<code>docker stop name</code>	Stop it (still exists, just not running)
<code>docker start name</code>	Start a stopped container again
<code>docker rm name</code>	Delete a container (<code>-f</code> forces it even if running)
<code>docker container prune</code>	Delete ALL stopped containers at once

💡 Bonus detail — `-d` and why hello-world "vanished"

`-d` (detached) runs a container in the background and frees my terminal. Without it, the container runs in the foreground and ties up the terminal. Also: a container lives only as long as its main program runs. nginx stays up because the web server keeps running; `hello-world` printed its message and immediately **exited** — which is why it showed as *"Exited (0)"* in `docker ps -a`. **Exit code 0 = finished cleanly** (non-zero = error).

6 Gotchas I Actually Hit Today

🚫 Gotcha 1 — "No such container: hello-world"

I tried `docker rm -f hello-world` and it failed. Reason: **hello-world is the IMAGE name, not the container name**. The container had an auto-generated name (*magical_franklin*).

Fix: use the name from the *NAMES* column, or the container ID.

Remember: `docker rm` removes a **container**; `docker rmi` removes an **image**.

🚫 Gotcha 2 — short names don't work

`docker stop ros` failed; `docker stop roshanprac` worked. Docker needs the **full** container name (or enough of the container *ID* to be unique) — it does not auto-complete partial names.

🚫 Gotcha 3 — the page still loaded after I stopped the server!

After `docker stop`, `localhost:8080` still showed the nginx page. The container was genuinely off — my **browser was showing a cached copy**. Browsers save pages to load them faster, so a normal refresh may never contact the server.

Honest test: use `curl http://localhost:8080` (curl never caches) — a stopped container gives *"Connection refused"*. Or hard-refresh with **Cmd+Shift+R**.

⚠️ Lesson: a browser showing a page ≠ the server is up

When checking whether a container is really serving, trust `curl` or a hard refresh — never a plain browser refresh. This fools real developers constantly.

7 Things I Didn't Ask — But Should Know

💡 Image layers & build caching

A Dockerfile builds an image in **layers** — one per instruction. Docker caches them, so the second build of the same thing is almost instant. That's why my Step-2 Dockerfile copies `package.json` and runs `npm install` *before* copying the rest of the code: as long as dependencies don't change, Docker reuses the cached install instead of re-downloading.

💡 `-it` — getting inside a container

`docker exec -it <name> sh` opens a shell *inside* a running container, like SSH-ing into it. `-it` = interactive + terminal. Great for poking around and debugging.

💡 Images vs containers — separate cleanups

Stopped containers and unused images both eat disk. `docker container prune` clears stopped containers; `docker image prune` clears dangling images; `docker system prune -a` sweeps both (careful — removes all unused images). My `docker info` showed 9 stopped containers from old projects — worth pruning.

💡 Naming & `--rm`

If I don't pass `--name`, Docker invents a random one (e.g. `magical_franklin`). Passing `--name` makes containers easy to reference. Adding `--rm` to `docker run` auto-deletes the container when it exits — handy for throwaway test runs so they don't pile up.

💡 Re-using a name

You can't have two containers with the same name. Trying to `docker run --name roshanprac` again while one exists errors with *"name already in use."* Remove the old one first: `docker rm -f roshanprac`.

8 Why Learn Docker in the AI Era?

If AI can write Docker files for me, why learn it? Because AI removes the *typing*, not the **understanding** — and understanding is the actual job.

✅ Added — the honest answer

AI can generate a `docker-compose.yml` in seconds. But when it **doesn't start** (port conflict, wrong env var, a service that can't reach Redis), I have to read the logs and know what *"Connection refused"*

means — exactly what I learned today. AI also hallucinates flags and gives outdated answers; I need the judgment to catch that.

The old skill (commoditized)	The skill that matters now
Memorize every Docker flag	Know what's possible and what's correct
Write the YAML by hand	Review, debug & verify what AI writes
Be the typist	Be the architect and the judge

- **Docker is infrastructure, not a fad** — nearly every backend job, cloud deploy and CI pipeline runs on containers.
- **AI itself runs in containers** — learning Docker is learning the ground the AI era is built on.
- **It's conceptual, not syntax** — isolation, images, networking, ports. That understanding transfers everywhere and doesn't expire.

Bottom line

The valuable person isn't the one who can *type* the Docker command — it's the one who can look at what AI produced and say "**that's wrong, and here's why.**" I can only say that if I understand Docker. So I'm learning the exact thing that lets me **command** AI instead of blindly trusting it.

9 Every Command I Used Today

```
docker run hello-world           # first test
open -a Docker                   # start the engine (Docker
Desktop)
docker info                      # confirm engine is up
docker images                   # list local images
docker run -d -p 8080:80 --name roshanprac nginx # run nginx in background
docker run -d -P nginx           # auto-publish to a RANDOM host
port
docker ps                      # running containers
docker ps -a                   # all containers incl. stopped
docker logs roshanprac         # view output
docker stop roshanprac         # stop
docker start roshanprac        # start again
docker rm -f roshanprac        # force-remove a container
docker rmi nginx               # remove the image (rmi = remove
image)
docker container prune          # delete all stopped containers
curl http://localhost:8080      # honest test (no caching)
```

10 Where I Am on the Learning Ladder

✓ Step 0 — Docker runs

`docker run hello-world` worked. Engine confirmed.

✓ Step 1 — Ran a ready-made image (nginx)

Learned ports (incl. no- `-p` and `-P`), the lifecycle, image-vs-container, Docker Hub, and browser caching.

▶ Step 2 — Build my OWN image (NEXT)

A tiny Node/Express app + my own Dockerfile → `docker build` → run it. This is where Docker truly clicks.

3 Step 3 — Add one service with Docker Compose

Connect my app to Redis; learn services finding each other by name. (80% of real-world value lives here.)

4 Step 4 — Full multi-service stack

Frontend + backend + Redis (+ LiveKit last — it's the fiddly one).

✓ One-line summary of Day 1

I learned to **run** containers from ready-made images, control their **lifecycle**, understand **ports** (including what happens with no `-p` and with `-P`) and the **image-vs-container** distinction — saw why you test with `curl` not a cached browser — and understood why this knowledge matters even in the AI era. Next: build my own image.

